

CS620T: Compilers - Principles and Implementation

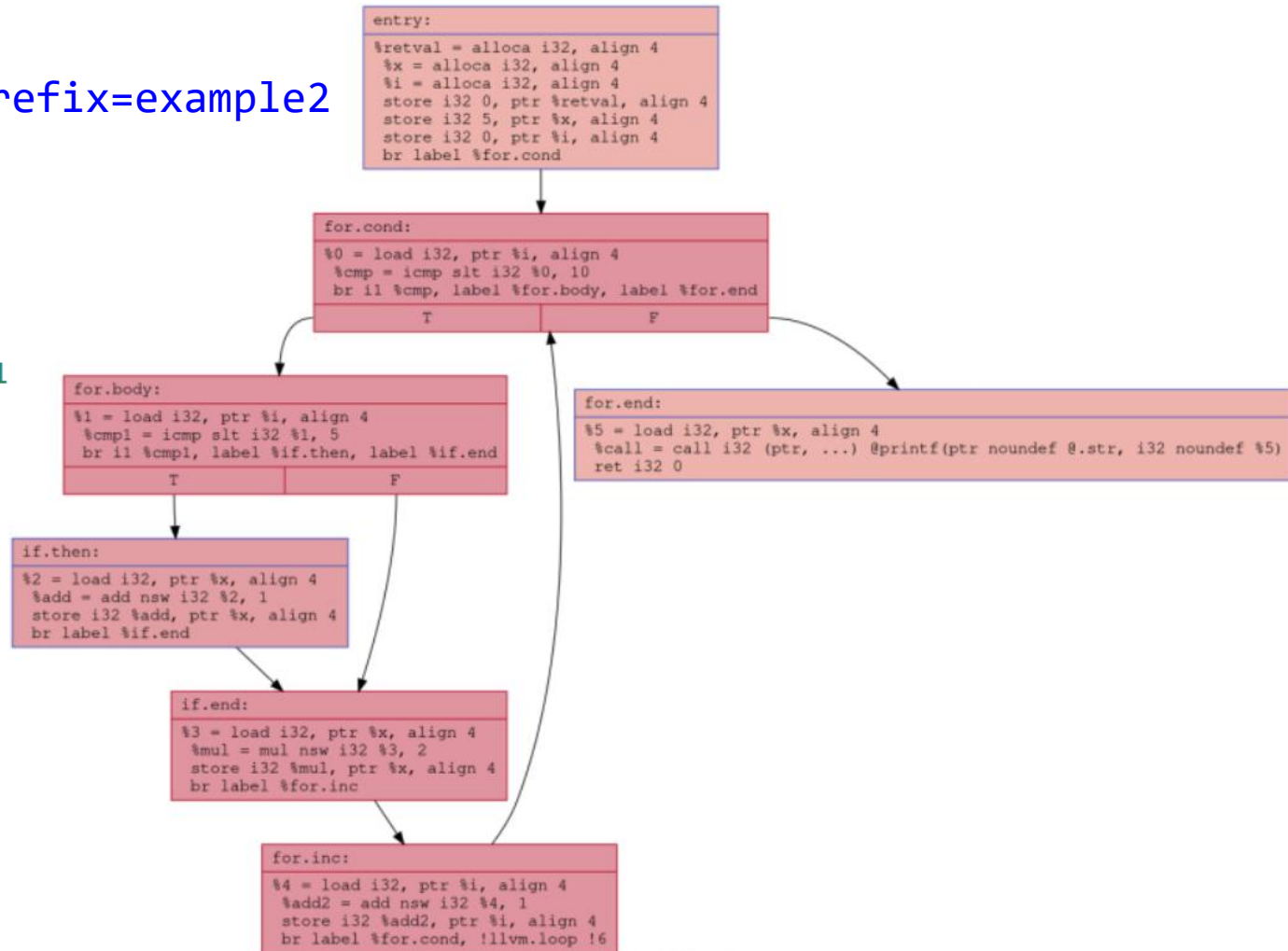
Autumn 2026

Week10: Demo of concepts learnt last week, Loop Optimizations

Flow Graphs - Representation

opt -passes=dot-cfg
-cfg-dot-filename-prefix=example2
example2.ll

```
3  int main() {  
4      int x, i;  
5      x = 5;  
6      for(i=0;i<10;i=i+1  
7          if (i < 5)  
8              x = x+1;  
9          x = x*2;  
10     }  
11     printf("%d",x);  
12     return 0;  
13 }
```

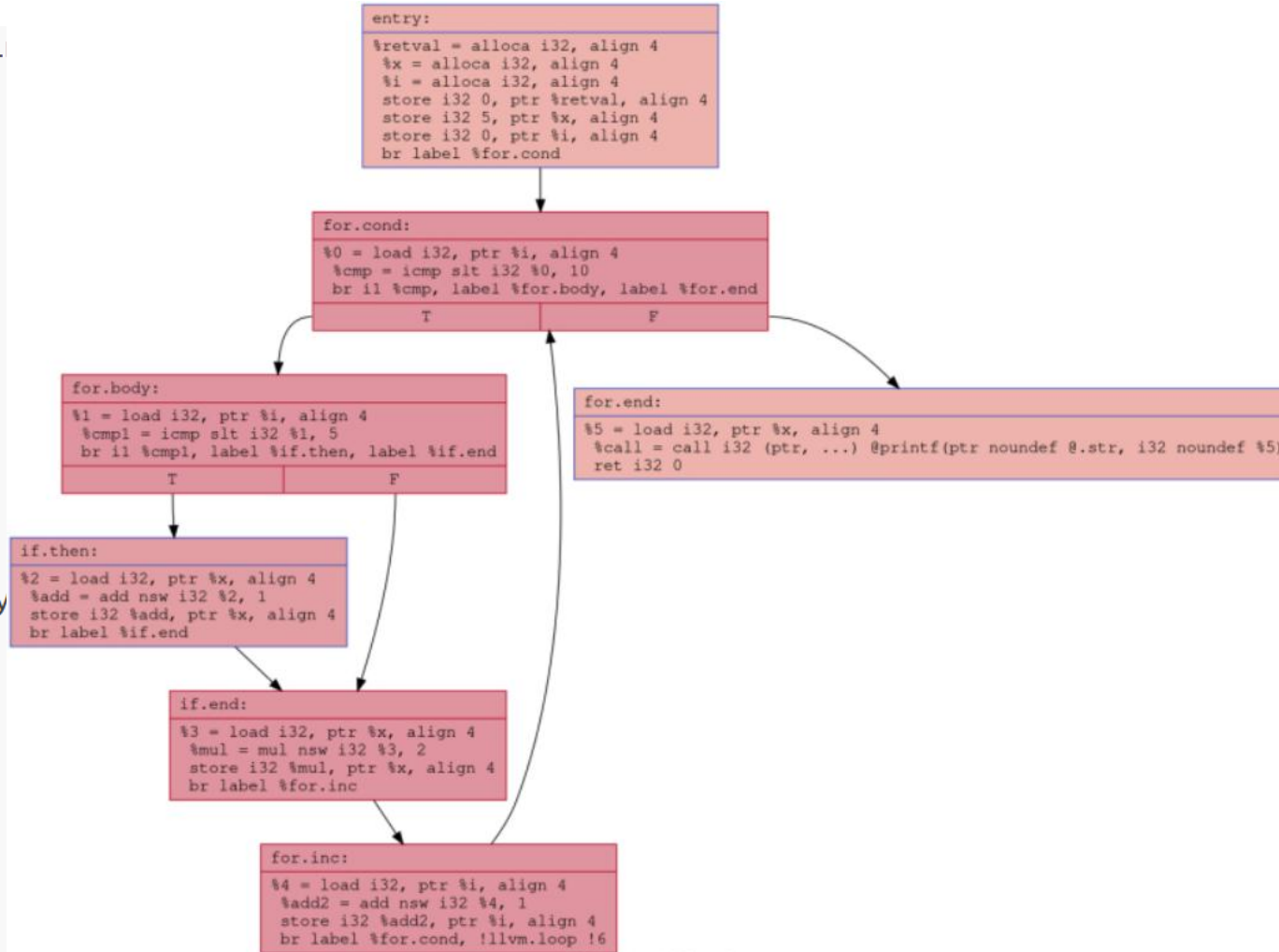


CFG for 'main' function

Dominators

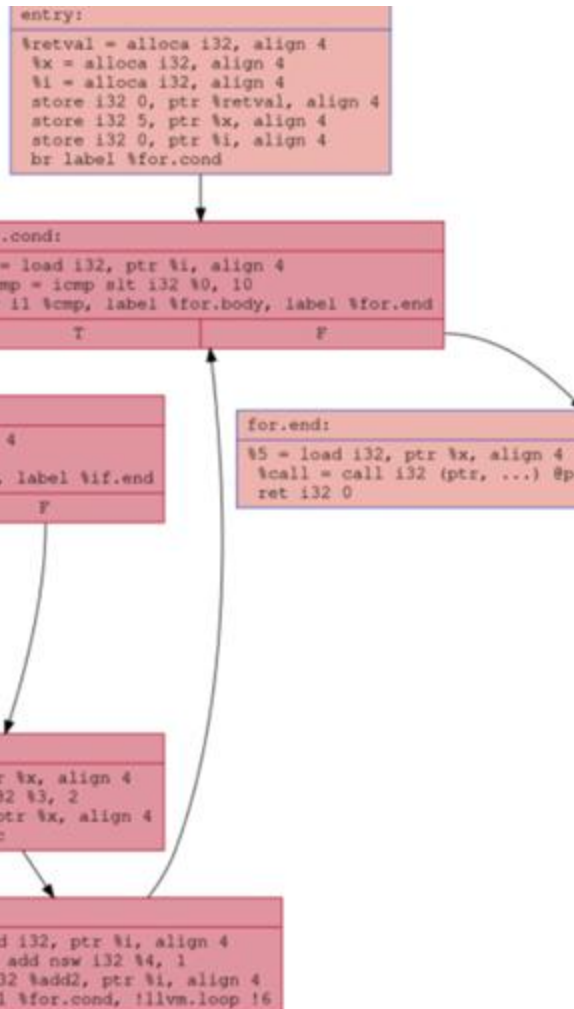
opt-load-pass-plugin=./fnmodpass/build/MyPass.so -passes="my-module-pass" -disable-output \$(TESTCASE).bc

[ModulePass] Function: mai
entry dominates entry
entry dominates for.cond
entry dominates for.body
entry dominates if.then
entry dominates if.end
entry dominates for.inc
entry dominates for.end
for.cond dominates for.cond
for.cond dominates for.body
for.cond dominates if.then
for.cond dominates if.end
for.cond dominates for.inc
for.cond dominates for.end
for.body dominates for.body
for.body dominates if.then
for.body dominates if.end
for.body dominates for.inc
if.then dominates if.then
if.end dominates if.end
if.end dominates for.inc
for.inc dominates for.inc
for.end dominates for.end

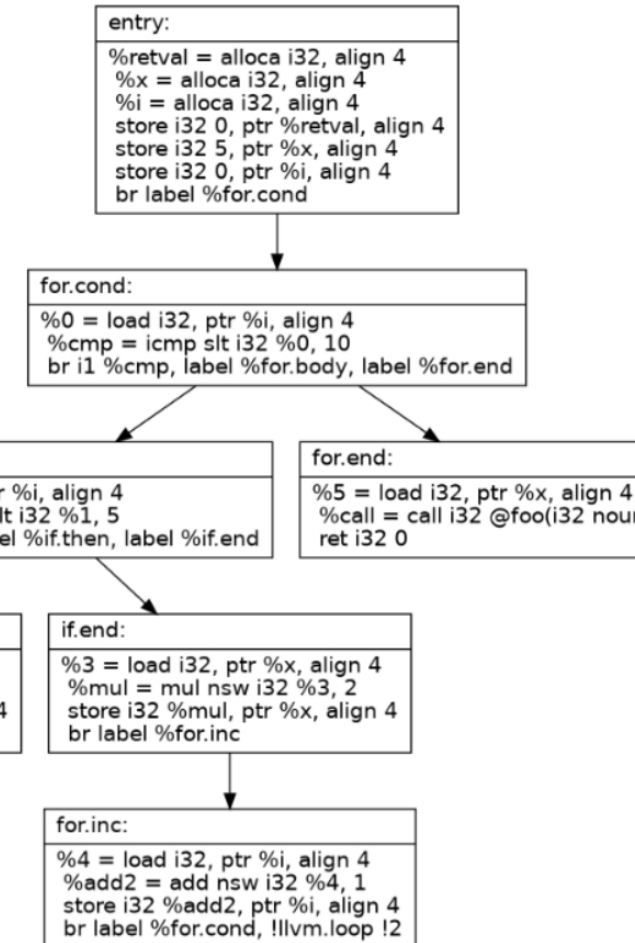


CFG for 'main' function

Dominator Tree



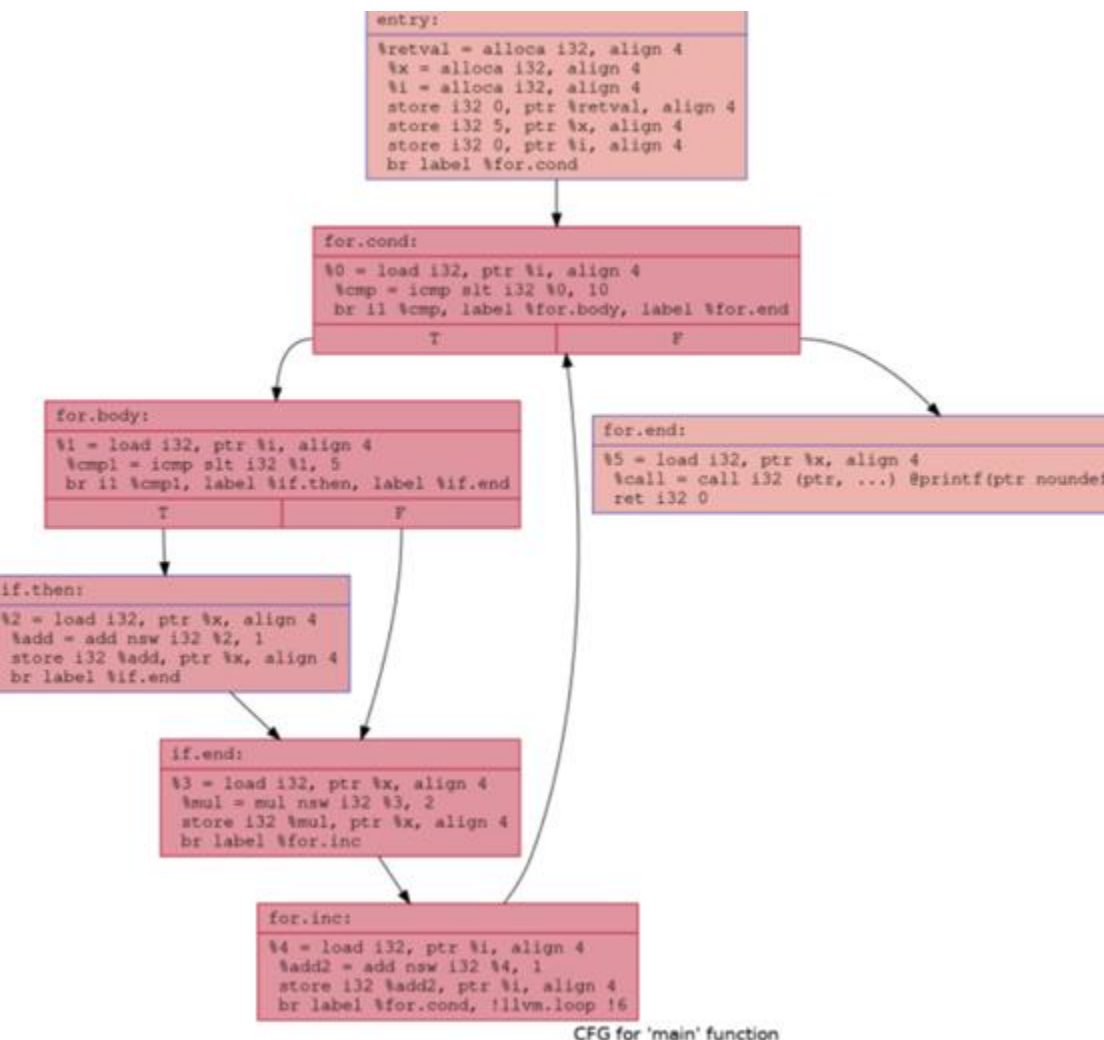
CFG for 'main' function



Dominator tree for 'main' function

opt -passes=dot-dom \$(TESTCASE).ll
 dot -Tpng dom.main.dot -o \$(TESTCASE)_dom.png

Finding Dominance Frontiers

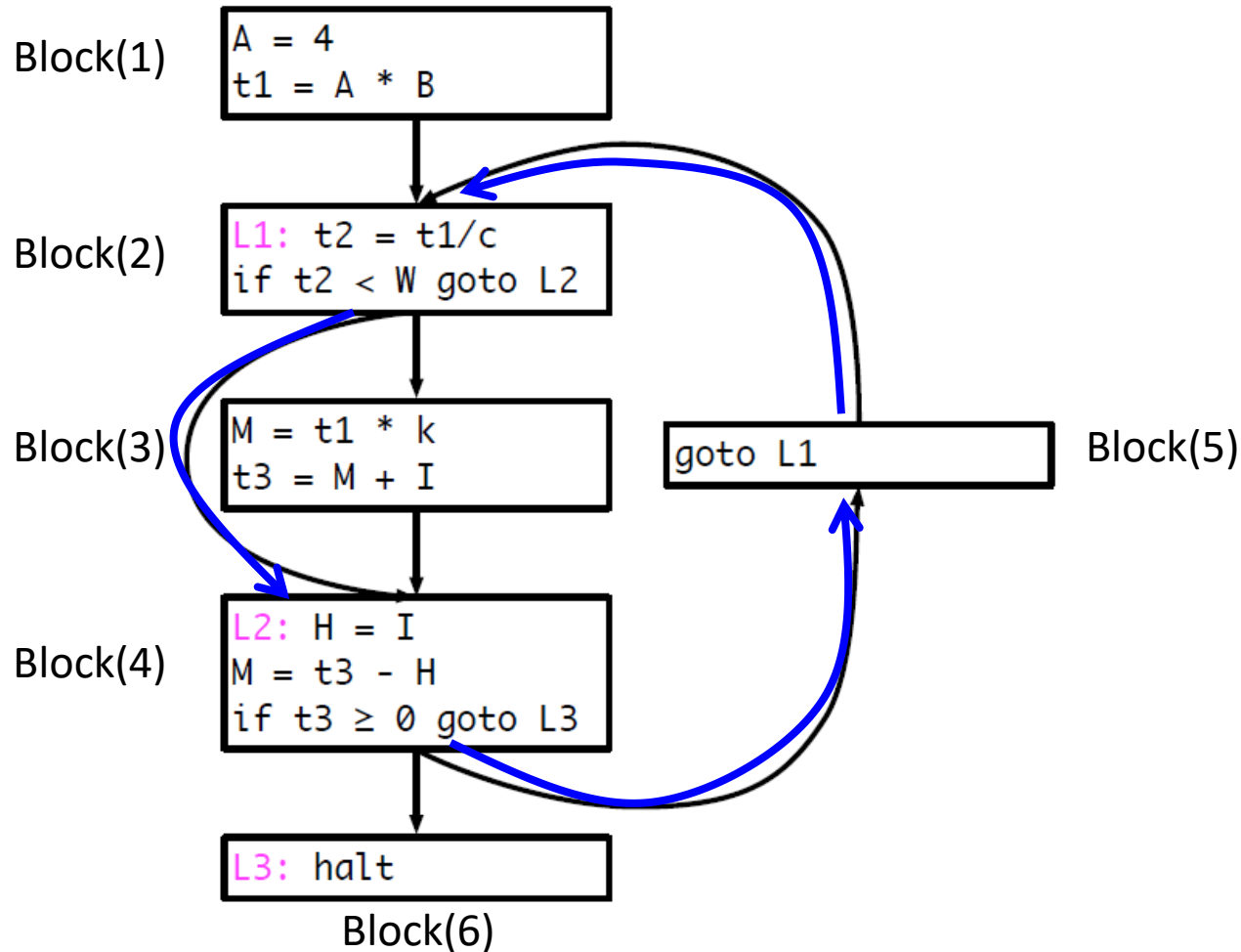


```

[ModulePass] Function: main
Function: main
Dominance frontier of Block entry:
  empty
Dominance frontier of Block for.cond:
  for.cond
Dominance frontier of Block for.body:
  for.cond
Dominance frontier of Block if.then:
  if.end
Dominance frontier of Block if.end:
  for.cond
Dominance frontier of Block for.inc:
  for.cond
Dominance frontier of Block for.end:
  empty
  
```

Note: `opt -passes=domtree ...`
 does not work as domtree is an internal pass

Exercise: Identify Loops in CFGs



Consider: {B2, B4, B5}. Is this a loop?, Are there other loops?

Loop Invariant Code Motion

- Optimization that works with natural loops
- Preheader – a unique block that is a predecessor of loop-entry block

Optimize Loops -Identifying Invariant Expressions

- How do we identify expressions that can be moved out of the loop?
 - LoopDef = { } set of variables defined (i.e. whose values are overwritten) in the loop body
 - LoopUse = { } 'relevant' variables used in computing an expression

```
Mark_Invariants(Loop L) {  
    1. Compute LoopDef for L  
    2. Mark as invariant all expressions,  
       whose relevant variables don't belong  
       to LoopDef  
}
```

Optimize Loops -Identifying Invariant Expressions

- Example

LoopDef{}

```
for I = 1 to 100      → {A, J, K, I}
  for J = 1 to 100    → {A, J, K}
    for K = 1 to 100  → {A, K}
      A[I][J][K] = (I*J)*K
```

Optimize Loops -Identifying Invariant Expressions

- Example

LoopUse{}

```
for I = 1 to 100      _____> {}  
  for J = 1 to 100    _____> {I}  
    for K = 1 to 100  _____> {I, J}  
      A[I][J][K] = (I*J)*K
```

Optimize Loops -Identifying Invariant Expressions

- Example

Invariant
Expressions

```
for I = 1 to 100
  for J = 1 to 100
    for K = 1 to 100 —————→ { I*J,
      A[I][J][K] = (I*J)*K          Addr(A[i][j]) }
```

For an array access, $A[m] \Rightarrow \text{Addr}(A) + m$

For 3D array above*, $\text{Addr}(A[I][J][K]) =$

$$\text{Addr}(A) + (I*10000) - 10000 + (J*100) - 100 + K - 1$$

Optimize Loops -Identifying Invariant Expressions

- Example

Invariant
Expressions

```
for I = 1 to 100
  for J = 1 to 100
    for K = 1 to 100
      A[I][J][K] = (I*J)*K
```

→ { Addr(A[i]) }

For an array access, $A[m] \Rightarrow \text{Addr}(A) + m$

For 3D array above*, $\text{Addr}(A[I][J][K]) =$

$$\text{Addr}(A) + (I*10000) - 10000 + (J*100) - 100 + K - 1$$

Optimize Loops -Factoring Invariant Expressions

- Move the invariant expressions identified

```
Factor_Invariants(Loop L) {  
    Mark_Invariants(L);  
    foreach expression E marked an invariant:  
        1. Create a temporary T  
        2. Replace each occurrence of E in L with T  
        3. Insert T:=E in L's header code  
           // If loop is known to execute at least once,  
           insert T:=E before LOOP:  
}
```

Optimize Loops -Factoring Invariant Expressions

- Example

```
for I = 1 to 100
  for J = 1 to 100
    for K = 1 to 100
      A[I][J][K] = (I*J)*K
```

//Invariant Expressions

Optimize Loops -Factoring Invariant Expressions

- Example

```
for I = 1 to 100
  for J = 1 to 100
    temp1=A[I][J]
    temp2=I*J
    for K = 1 to 100
      temp1[K] = temp2*K
```

Optimize Loops -Factoring Invariant Expressions

- Example

```
for I = 1 to 100
  temp3=A[I]
  for J = 1 to 100
    temp1=temp3[J]
    temp2=I*J
    for K = 1 to 100
      temp1[K] = temp2*K
```

Optimize Loops -Factoring Invariant Expressions

- Expressions cannot always be moved out!

Case I: We can move $t = a \text{ op } b$ if the statement dominates all loop exits where t is live

A node $bb1$ dominates node $bb2$ if all paths to $bb2$ must go through $bb1$

```
for (...) {  
    if(*)  
        a = 100  
}  
c=a
```

Cannot move $a=100$ because it does not dominate $c=a$ i.e. there is one path (when if condition is false) $c=a$ can be executed /'reached' without going to $a=100$

Optimize Loops -Factoring Invariant Expressions

- Expressions cannot always be moved out!

Case II: We can move $t = a \text{ op } b$ if there is only one definition of t in the loop

```
for (...) {  
    if(*)  
        a = 100  
    else  
        a = 200  
}
```

Multiple definition of a

Optimize Loops -Factoring Invariant Expressions

- Expressions cannot always be moved out!

Case III: We can move $t = a \text{ op } b$ if t is not defined before the loop, where the definition reaches t 's use after the loop

```
a=5
for (...) {
    a = 4+b
}
c=a
```

Definition of a in $a=5$ reaches $c=a$, which is defined after the loop

Optimize Loops

- Should be careful while doing optimization of loops

```
while J > I loop  
    A(j) := 10/I;  
    j := j + 2;  
end loop;
```

Optimize Loops – Code Motion

- Should be careful while doing optimization of loops

```
while J > I loop  
    A(j) := 10/I;  
    j := j + 2;  
end loop;
```

- Optimization: can move $10/I$ out of loop
- What if $I = 0$?
- What if $I \neq 0$ but loop executes zero times?

Optimization Criteria - Safety and Profitability

- **Safety** - is the code produced after optimization producing same result?
- **Profitability** - is the code produced after optimization running faster or uses less memory or triggers lesser number of page faults etc.

```
while J > I loop  
    A(j) := 10/I;  
    j := j + 2;  
end loop;
```

- E.g. moving I out of the loop introduces exception (when I=0)
- E.g. if the loop is executed zero times, moving $A(j) := 10/I$ out is not profitable

Converting to SSA form

- Where do we insert phi nodes?

```
for v in vars {  
  for d in defs[v] {  
    for block in DF[d] {  
      if (block does not have a phi node)  
        add phi node to block  
      if (block is not part of defs[v])  
        add block to defs[v]  
    }  
  }  
}
```

Converting to SSA form

- Rename variables

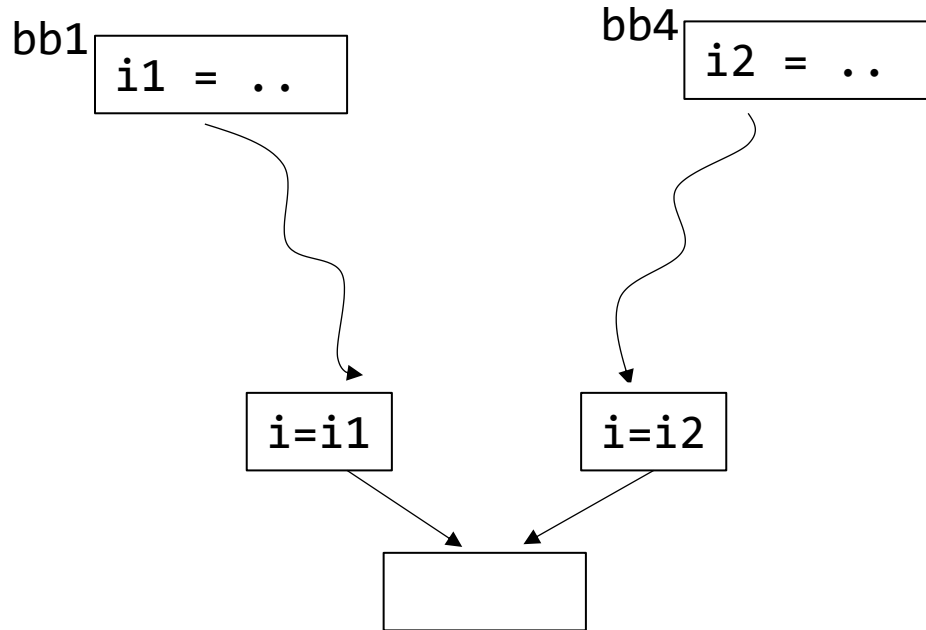
Stack[v] is a stack of variable names for every variable v

```
Rename(block) {  
    foreach instn in block {  
        replace each argument to instn with stack[argument's oldname]  
        replace instn's destination with newname  
        push newname to stack[destination's oldname]  
    }  
  
    foreach s in successor(block){  
        foreach p in s's phi nodes {  
            read stack[v] and plug in p (if p is for v)  
        }  
    }  
  
    foreach b in immediately_dominated_by(block)//need dominance tree here  
        Rename(b)  
  
    pop all names pushed to stack  
}
```

```
Rename(entry)
```

Converting **from** SSA form

- Get rid of Phi nodes



Acknowledgements

- [Compiler Explorer](#) (browser-based option to view LLVM IR with phi nodes.
Use `-O1 -emit-llvm` compiler flag with `phi.c`)
- https://www.llvmpy.org/llvmpy-doc/dev/doc/llvm_concepts.html
- [CS 6120: The Self-Guided Course](#) (lesson 5)